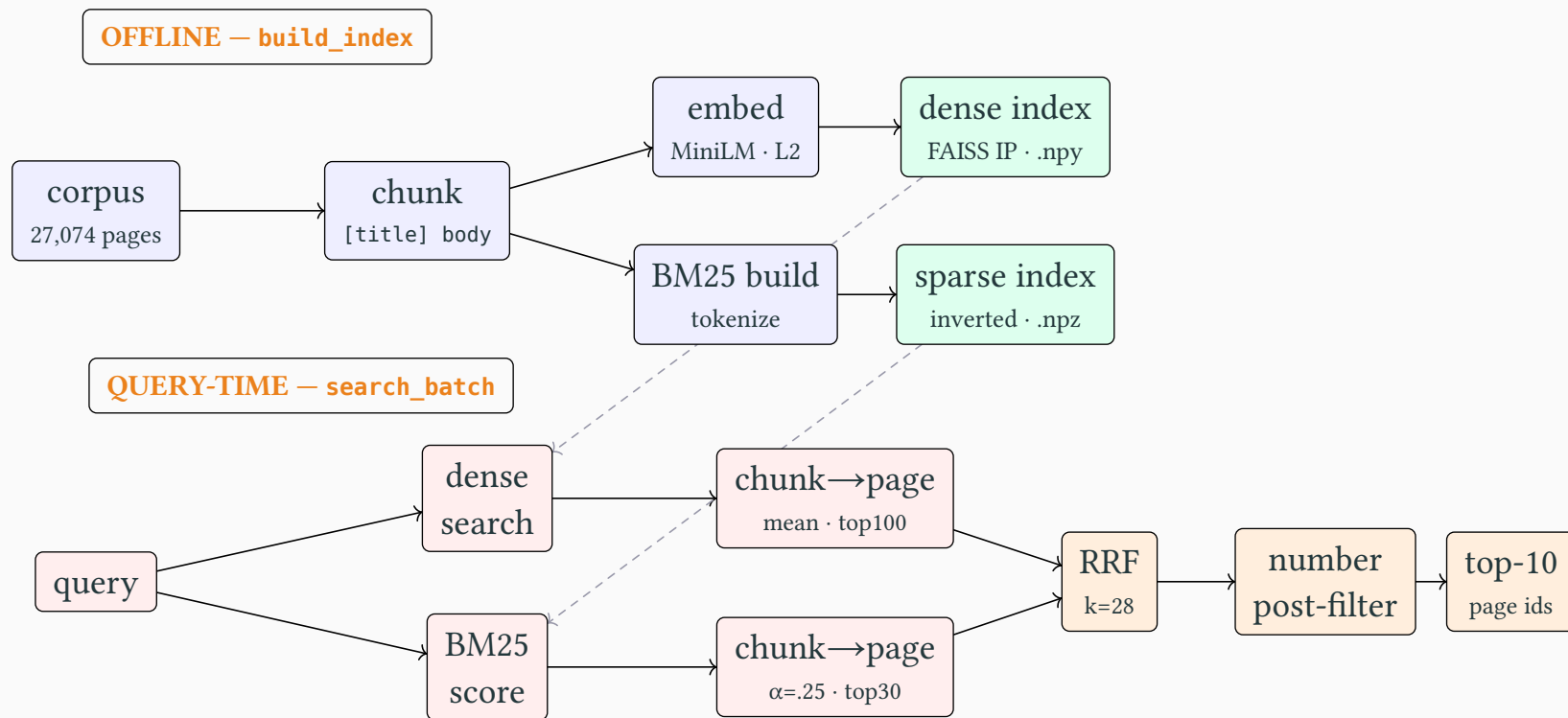


Project A — Section B: Retrieval Pipeline

End-to-end retrieval over 27k Wikipedia-style pages

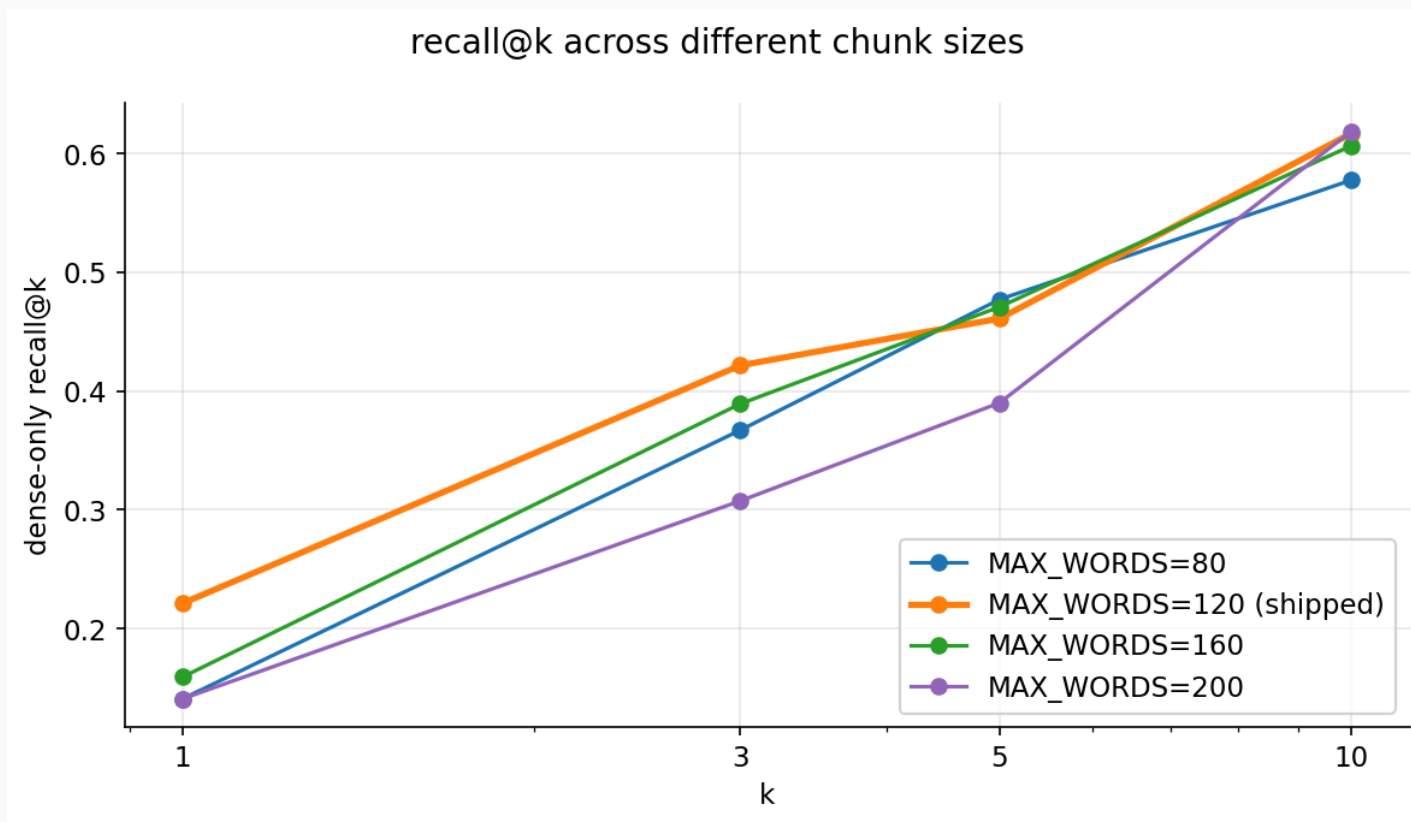
Slide 1 — Pipeline



Final results: NDCG@10 = 0.53

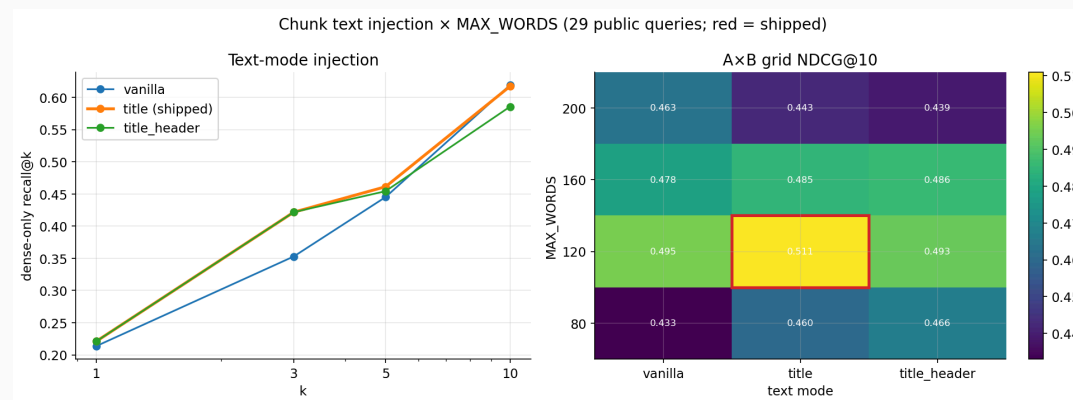
- **chunking strategy:** sliding window + no-overlap per paragraphs
- Wikipedia articles are *already* split semantically
- file `header.py` used to extract paragraph headers deterministically, using heuristics:
 - Search for short sentences (≤ 6 words) before double line breaks `\n\n`
 - Leading capital, ends with a period, no quotes.
 - Reject any phrase carrying an auxiliary/pronoun token (is, was, it, they...)
- proved to be faster and more effective than semantic embeddings.
- Used `recall@k` ($k \in [1, 10]$) to measure embedding effectiveness:
 - tested with different overlap window sizes
 - tested with different `max_words` per chunk - a heuristic for the number of tokens

Slide 2 — chunk.py: max_words sweep



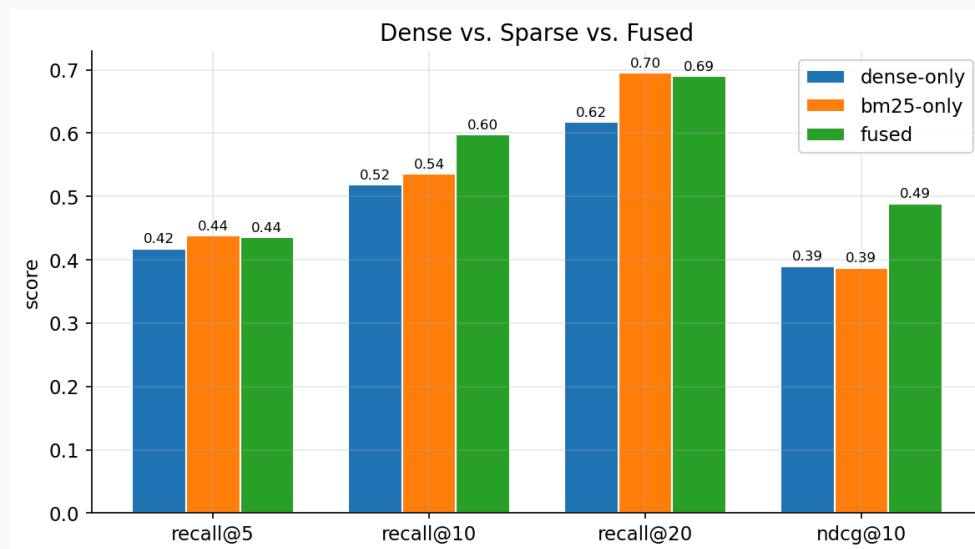
Slide 2 — chunk.py: title injection

- We have found that some chunks lack context to the overall theme.
- Solution: inject the title into every chunk: [TITLE]: BODY
- Tested across the validation set with:
 - original chunk
 - injected article title
 - injected article title + paragraph title



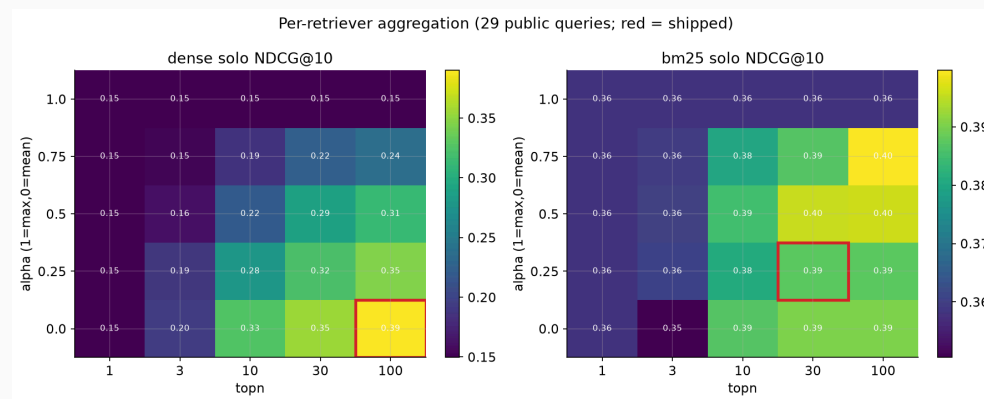
- No changes were made from the original implementation
- Optimized batch size for faster GPU run times.

- Artifacts:
 - Dense all-MiniLM-L6-v2 embeddings
 - metadata saved: exact numbers (used later for filtering), titles, paragraphs.
 - Sparse index (BM25)
- Implemented a classic BM25 index to work alongside the dense embeddings
 - Per query term: score tf-idf , summed over chunk.
 - Applied length normalization $B=0.5$
- Validated on sparse vs dense vs fused



Slide 6 — retrieve.py

- Each index aggregates its chunk scores up to a page score with a max/mean blend (alpha: 1=max, 0=mean).
- Grid search: find optimal alpha and top_n values for each index.
- Final hyperparameters:
 - Dense → **mean** ($\alpha = 0.0$, top_n=100).
 - BM25 → **near-mean** ($\alpha = 0.25$, top_n=30).



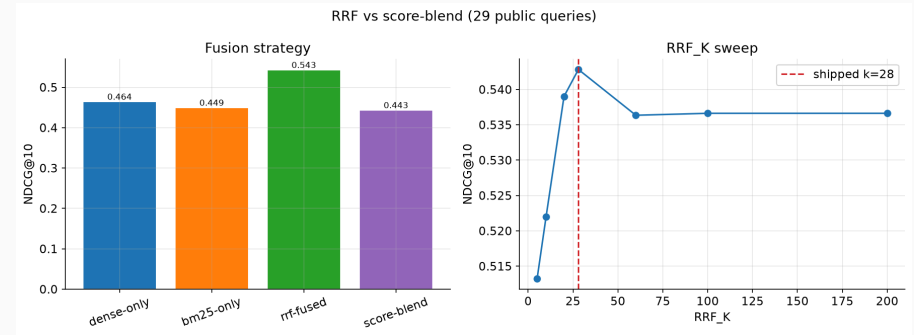
Slide 7 — retrieve.py: Reciprocal Rank Fusion

- **Reciprocal Rank Fusion (RRF)**: combine the two ranked lists using **rank position only**,

$$\text{score}(p) = \sum_{r \in R} \frac{1}{k + \text{rank}_r(p) + 1}$$

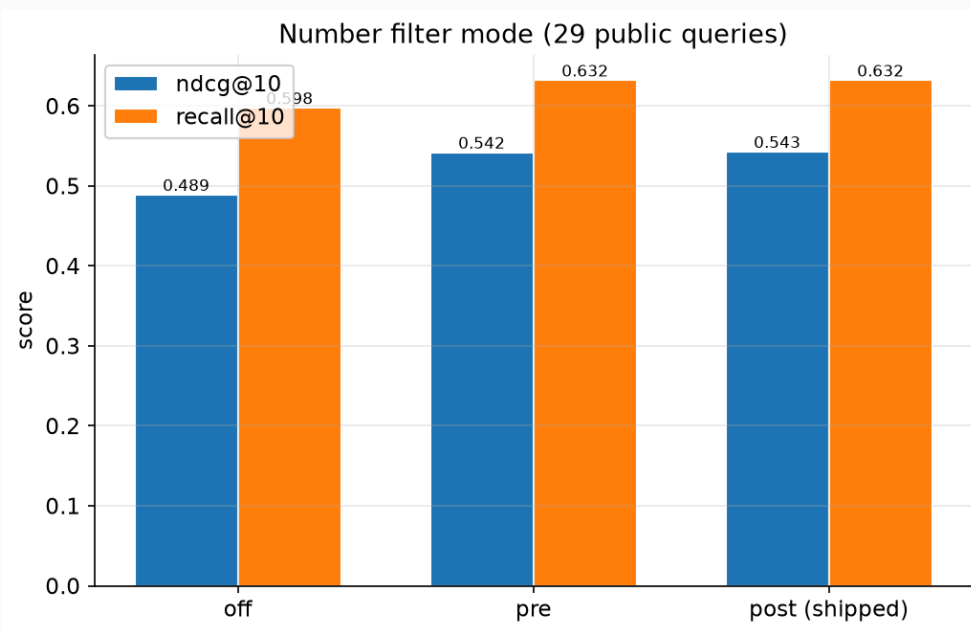
(sum over each retriever list r , with $k = 28$)

- Each retriever returns an independently ranked page list. A page at rank r in a list contributes $1/(k + r + 1)$ to its fused score
- Asymmetric depth:
 - dense contributes its top **100**
 - sparse contributes its top **60** (SPARSE_RRF_DEPTH)
- pages re-sorted by total score.



Slide 7 — retrieve.py: Post-filtering

- During chunking, numbers are saved to the metadata.
- Queries containing numbers will trigger a post-filtering of the final list.
 - Years are expanded: “... in the 1920s” => expanded to {1920, 1921, ..., 1929}
- post mode: rank the full corpus first, only then drop non-matching pages



Slide 8 - Evaluation

We used the following methods to avoid overfitting for the `public_queries.json` provided validation set:

- Public dataset enrichment
 - Extracted data from the SQuAD v2 dataset - an IR benchmark dataset
 - Created a hybrid set of queries by matching the Wikipedia titles in SQuAD to entries of the provided dataset
 - We reached $\text{NDCG@10} = 0.42$ for 120 queries with a single answer.
- LLM Enrichment:
 - Used a short script to pick some articles at random from the corpus
 - prompted an LLM to create a query fitting of that article
 - verified the validity of the queries and saved as JSON.

Final scores across all datasets (mean NDCG@10 , shipped fusion, $\text{RRF_K}=28$):

Dataset	Queries	mean NDCG@10
public_queries (provided dev set)	29	0.5297
llm_queries (LLM-generated)	30	0.8597
squad_queries (SQuAD v2 hybrid)	119	0.4618